

Regex

Exemple 1 : `[A-Z]{3}[0-9]{3}`

Signifie :

3 majuscules, suivies de 3 chiffres.

Donc `ABC123` correspond, mais `A1BC23` ne correspond pas.

En contexte HTML `pattern`, cela signifie que **toute la valeur** doit correspondre à ce motif.

Exemple 2 : `[A-Z][0-9][^A-Za-z0-9]{9,}`

Signifie :

une majuscule soit en premier, puis un chiffre, puis un caractère spécial (ni lettre ni chiffre) et **au moins 9 caractères supplémentaires**.

Donc il faut **au moins 12 caractères au total**.

`A1!abcdefghij` fonctionne effectivement : il contient 14 caractères.

`bonjour1!Axyz` ne correspond pas, car il commence par `b` et non par une majuscule.

Exemple 3 : `^(?=.*[A-Z])(?=.*[0-9])(?=.*[^A-Za-z0-9]).{12,}$`

Cette regex sert typiquement à vérifier qu'un mot de passe respecte plusieurs contraintes.

Partie	Signification
<code>^</code>	Début de la chaîne
<code>(?=.*[A-Z])</code>	Il doit y avoir au moins une majuscule
<code>(?=.*[0-9])</code>	Il doit y avoir au moins un chiffre
<code>(?=.*[^A-Za-z0-9])</code>	Il doit y avoir au moins un caractère qui n'est ni une lettre ni un chiffre (<code>!</code> , <code>@</code> , <code>#</code> , etc.)
<code>.</code>	N'importe quel caractère (sauf, généralement, un saut de ligne)
<code>{12,}</code>	Au moins 12 caractères
<code>\$</code>	Fin de la chaîne

Les `(?=.*...)` : les lookaheads

Les trois expressions suivantes sont des **lookaheads positifs** :

```
(?=.*[A-Z])  
(?=.*[0-9])  
(?=.*[^A-Za-z0-9])
```

Ils vérifient une condition **sans consommer de caractères**.

Par exemple : `(?=.*[A-Z])` se lit :

« À partir d'ici, je dois pouvoir trouver quelque part une lettre entre **A** et **Z**. »

Le `.*` signifie « zéro ou plusieurs caractères quelconques », puis `[A-Z]` impose la présence d'une majuscule.

Le caractère spécial `[^A-Za-z0-9]`

Le `^` à l'intérieur des crochets signifie « négation ».

Donc : `[A-Za-z0-9]` = une lettre ou un chiffre

tandis que : `[^A-Za-z0-9]` = quelque chose qui **n'est ni une lettre ASCII ni un chiffre**.

Cela peut donc être `!`, `@`, `#`, `$`, `-`, `_`, un espace, etc.

Remarques

Cette regex ne signifie pas forcément « 12 caractères comprenant une lettre, un chiffre et un symbole » au sens Unicode.

Par exemple, `[A-Z]` ne reconnaît que les **majuscules ASCII** : **A** à **Z**. Il ne reconnaîtra pas **É**, **À**, **Ç**, etc. De même, `[^A-Za-z0-9]` = quelque chose qui **n'est ni une lettre ASCII ni un chiffre ASCII**.

Dans l'attribut HTML `pattern`, les ancres `^` et `$` sont redondantes, car le navigateur vérifie déjà que **toute la valeur** correspond au pattern.

On peut donc écrire simplement :

```
pattern="(?!.*[A-Z])(?!.*[0-9])(?!.*[^A-Za-z0-9]).{12,}"
```